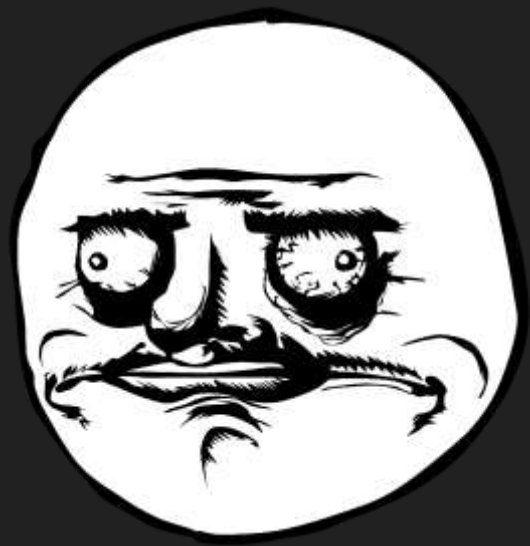


Git y Github básico

@nelsonkewebs

Temario.

1. ¿Que es git y github?
2. Un poco de historia.
3. Instalación de git.
4. Configuración básica.
5. Algunos comandos y para que funcionan.
6. Los tres estados.
7. Buenas prácticas.
8. Fuentes.



1. ¿Qué es Git y Github?



<https://git-scm.com/images/logos/downloads/Git-Logo-2Color.png>



<https://i1.wp.com/linuxnewbieguide.org/wp-content/uploads/2017/03/github-logo.png?ssl=1>

1. ¿Qué es git y github?

“Git es un sistema de control de versiones distribuidas de código abierto y gratuito diseñado para manejar todo, desde proyectos pequeños a muy grandes, con velocidad y eficiencia.”

<https://git-scm.com/>

“GitHub es una plataforma de desarrollo inspirada en la forma en que trabajas. Desde el código abierto hasta el negocio, puede alojar y revisar códigos, administrar proyectos y crear software junto a millones de otros desarrolladores.”

<https://github.com/>

1. ¿Qué es git y github?

Pero, ¿qué es el control de versiones?

“El control de versiones es un sistema que registra los cambios en un archivo o conjunto de archivos a lo largo del tiempo para que pueda recuperar versiones específicas más tarde.”

ProGit.

2. Un poco de historia.

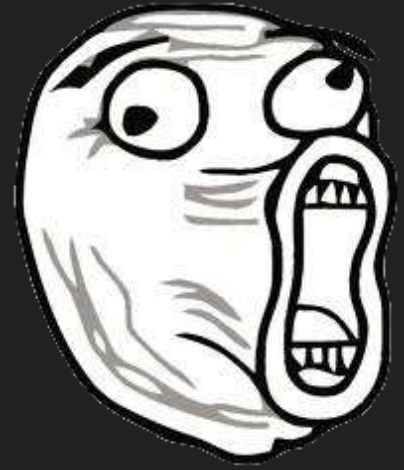


2. Un poco de historia.

Git fue desarrollado por uno de los grandes en la historia del software libre, **Linus Torvalds**, quien buscaba un sistema de control de versiones como BitKeeper (y Monotone, bases de git) que fuera gratuito y eficiente, sin embargo, en el mercado no existía algo como tal. Aunque Git, en un principio no fue exactamente un sistema de control de versión, la comunidad de desarrolladores (principalmente) adoptaron rápidamente esta tecnología gracias a sus diferentes cualidades tales como Gestión distribuida, Gestión eficiente de proyectos grandes, utilización de repositorios, entre otros.

Lo que vino a mejorar la manera arcaica en la que se crean carpetas con el nombre del proyecto concatenado a una fecha y dentro copia de todos los archivos para irlos almacenando como versiones del mismo.

3. Instalación de git.



3. Instalación de git.

Instalación de git en Linux

Debian/Ubuntu

apt-get install git

Fedora

yum install git

Alpine

apk add git

Instalación de git en Windows

Basta con descargar desde la página oficial, el ejecutable de instalación y siguiente, siguiente, ...

Instalación de git en Mac

Basta con descargar desde la página oficial, el ejecutable de instalación y siguiente, siguiente, ...

Dirección para descarga de git: <https://git-scm.com/downloads>

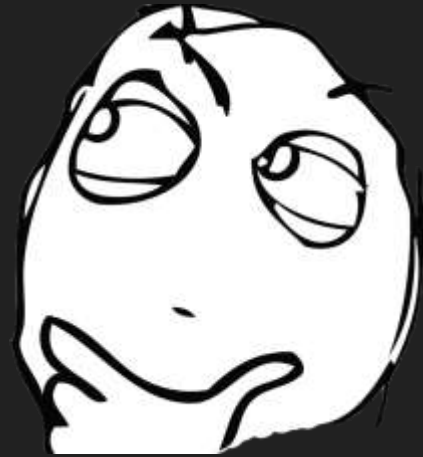
3. Instalación de git.

Red Hat Enterprise Linux, Oracle Linux, CentOS, Scientific Linux, et al.

Para sistemas Red Hat y derivados debes descargar el código fuente, configurar, compilar e instalar.

Dirección para descarga el código fuente de git: <https://www.kernel.org/pub/software/scm/git/>

4. Configuración básica.



4. Configuración básica.

Antes de empezar a utilizar git, debes de hacer la siguiente configuración:

Identificate

```
$ git config --global user.name "Nelson Manuel"
```

```
$ git config --global user.email nelsonmanuel@outlook.com
```

```
$ git config user.name
```

```
Nelson Manuel
```

```
$ git config user.email
```

```
nelsonmanuel@outlook.com
```

4. Configuración básica.

¿Quieres ver más?

```
$ git config --list
```

```
user.name=Nelson Manuel
```

```
user.email=nelsonmanuel@outlook.com
```

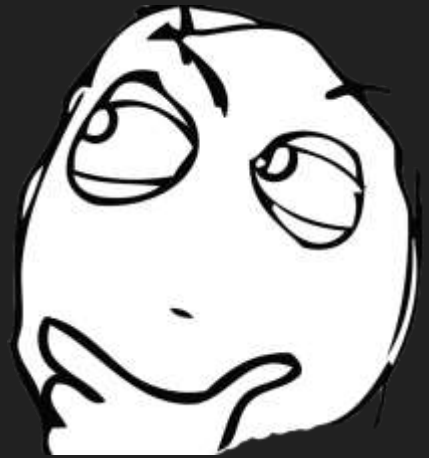
```
color.status=auto
```

```
color.branch=auto
```

```
color.interactive=auto
```

```
color.diff=auto
```

5. Algunos comandos
y para que funcionan.



5. Algunos comandos y para que funcionan.

`$ git init`

Inicializa un repositorio vacío.

`$ git status`

Muestra el estado actual de la rama, como los cambios que hay sin commitear.

`$ git add .`

Comienza a trackear los archivos con cambios.

`$ git commit -m "mensaje"`

Registra un commit en el repositorio con un "mensaje".

`$ git commit -am "mensaje"`

Confirma los cambios realizados. El "mensaje" generalmente se usa para asociar al commit una breve descripción de los cambios realizados.

5. Algunos comandos y para que funcionan.

`$ git branch`

Muestras la ramas existentes y en cual estamos actualmente. Pasando un nombre, se crea una rama.

`$ git log`

Muestra los commits realizados.

`$ git checkout commit_id or branch_name`

Se ubica en la versión que se registró en el commit o se hace el cambio de rama

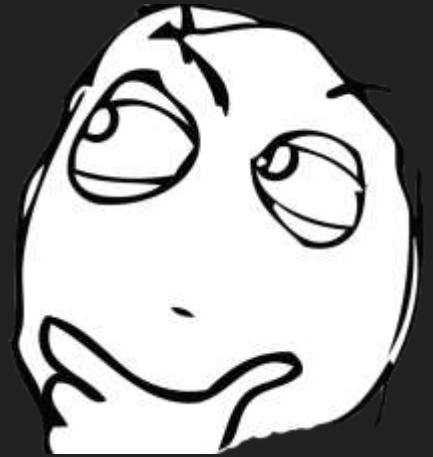
`$ git diff branch_name_one branch_name_two`

Registra un commit en el repositorio con un “mensaje”.

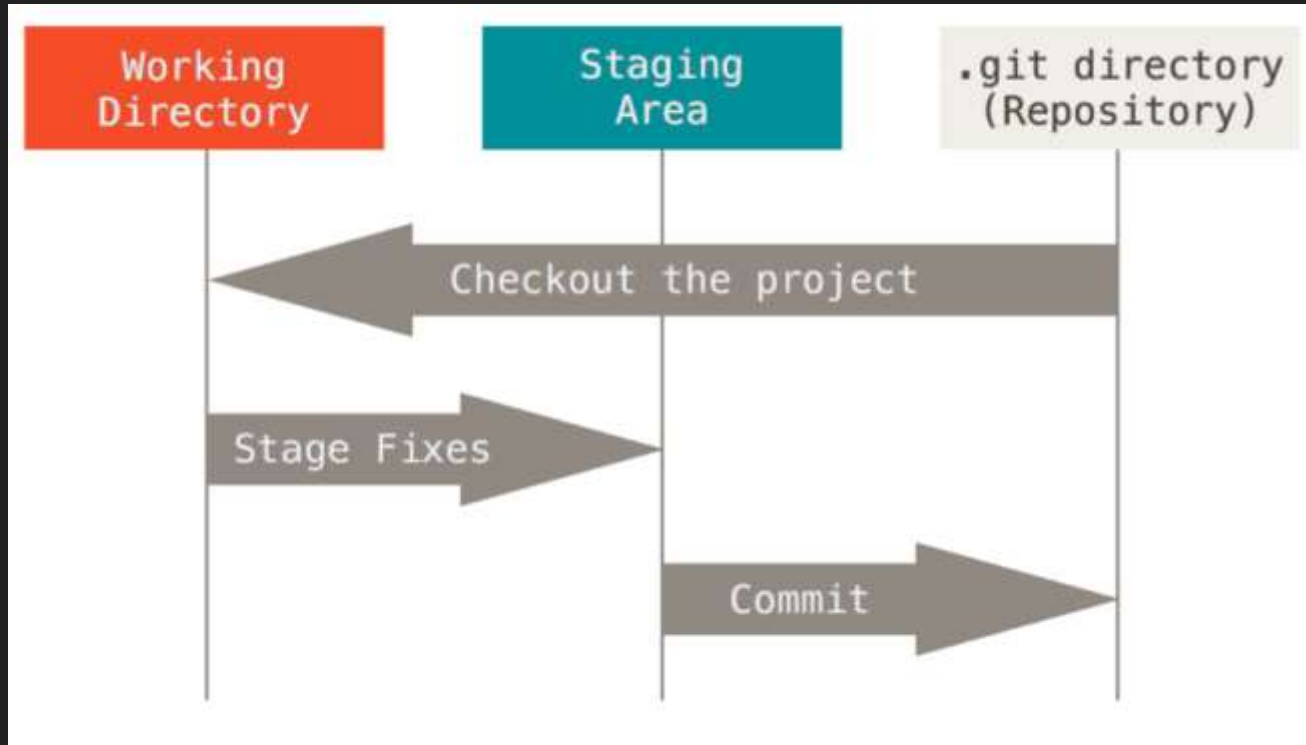
`$git clone URL`

Clona un repositorio remoto en el directorio local.

6. Los tres estados.



6. Los tres estados.



7. Buenas prácticas.



7. Buenas prácticas.

Se deben utilizar 4 tipos de ramas: Master, Development, Features, y Hotfix.

Master:

Es la rama principal. Contiene el repositorio que se encuentra publicado en producción, por lo que debe estar siempre estable.

Development:

Es una rama sacada de master. Es la rama de integración, todas las nuevas funcionalidades se deben integrar en esta rama. Luego que se realice la integración y se corrijan los errores (en caso de haber alguno), es decir que la rama se encuentre estable, se puede hacer un merge de development sobre la rama master.

7. Buenas prácticas

Features:

Cada nueva funcionalidad se debe realizar en una rama nueva, específica para esa funcionalidad. Estas se deben sacar de development. Una vez que la funcionalidad esté desarrollada, se hace un merge de la rama sobre development, donde se integrará con las demás funcionalidades.

Hotfix:

Son bugs que surgen en producción, por lo que se deben arreglar y publicar de forma urgente. Es por ello, que son ramas sacadas de master. Una vez corregido el error, se debe hacer un merge de la rama sobre master. Al final, para que no quede desactualizada, se debe realizar el merge de master sobre development.

Fuente: <https://es.wikipedia.org/wiki/Git>

8. Fuentes.

1. <https://es.wikipedia.org/wiki/Git>
2. <https://github.com/>
3. <https://github.com/>
4. [ProGit Second Edition, Scott Chanco and Ben Straub, Apress.](#)